ChatGPT

Personalized Recommendation Layer for JioHotstar

Hotstar Streaming Architecture and APIs

【53†embed_image】JioHotstar (formerly Hotstar) uses a highly distributed microservices architecture on AWS to serve millions of concurrent users. User requests hit Hotstar's global CDN (e.g. Akamai/CloudFront) which acts as an API gateway, then flow through AWS Application Load Balancers (ALBs) into Kubernetes (EKS) node pools【1†L124-L133】【3†L307-L315】. Engineers note that Hotstar broke its monolith into ~800 backend services (video, login, ads, recommendations, etc.) each with its own ALB【14†L109-L117】【3†L307-L315】. The platform employs extensive caching (for static home-page modules and sports scores) and adaptive-bitrate (HLS/DASH) streaming to handle peak loads【14†L109-L117】. For example, Hotstar even created a separate CDN domain for cacheable API responses, lightening security for static content and freeing compute capacity【3†L174-L179】. All critical calls (login, "watch now," personalized recs) are prioritized, while non-essential modules load asynchronously【14†L109-L117】.

- **No public API:** Hotstar does *not* publish a public developer API for movie/show data. (All data APIs are internal to their apps.) Community developers have reverse-engineered private endpoints, but these require Hotstar auth tokens. For instance, a Kodi plugin calls an internal JSON endpoint `/o/v2/menu` (with Android platform headers) to fetch menu items【42†L930-L938】. In practice we assume no official API – any data must come from web-scraping, partner catalogs, or similar.

Cataloging JioHotstar Content

To know what movies/shows exist on Hotstar, we rely on third-party catalogs and scraping. For example, 91mobiles maintains a comprehensive Hotstar movie list – currently citing about 4317 movies and 1682 TV shows on Hotstar【16†L171-L174】. Such sites (and others like JustWatch or Reelgood) can be scraped or queried to build our master catalog. Similarly, Watchmode's Streaming Availability API covers 200+ services worldwide【24†L37-L45】 – and explicitly lists Hotstar as a provider【26†L279-L287】 – so it can tell us which titles are on Hotstar. In short, we can obtain a list of Hotstar titles (with IDs or names) from: third-party aggregators (91mobiles, Watchmode, RapidAPI services, etc.); scraped Hotstar web pages; or even commercial data providers. For instance, RealDataAPI advertises a "Jio Hotstar" scraper that extracts title, genre, release date, ratings, etc.【23†L186-L194】. Once we have the title list, we can augment it with metadata (genres, IMDb scores, posters, descriptions) via public movie APIs. Common choices are The Movie DB (TMDb) API or OMDb (IMDb) API – for example, one project used TMDb to fetch for each Hotstar title its overview, genres, ratings and poster【50†L128-L136】. In practice we would cross-reference Hotstar titles with TMDb/IMDb IDs to retrieve up-to-date info.

- **Example data sources:** 91mobiles' Hotstar pages (scrapable HTML with 4000+ titles)【16†L171-L174】; Watchmode API (returns provider catalogs)【24†L37-L45】【26†L279-L287】; TMDb or

OMDb APIs for movie metadata【50†L128-L136】; and even crowdsourced "hotstar API" scrapers【23†L186-L194】. With these we can validate the Hotstar catalog and keep it current.

Recommendation & Blocking UX Patterns

Modern streaming platforms recognize the need for user control over recommendations. **Allow easy hiding/removal:** UX experts advise that any "Continue Watching" or recommendation carousel should let users **remove items** they don't want【30†L100-L107】. For example, if a friend or mistake added a show to your resume list, you should be able to click "remove" so it won't clutter your profile【30†L100-L107】. Similarly, if you explicitly **dislike or block** something, it should disappear from future recs. (Netflix's mobile apps lack a "Not Interested" button, which frustrates users; by contrast, an HBO Max UI immediately lets you remove items from Continue Watching.)

- **Explicit feedback ("Not Interested"):** Another pattern is to let the user say "no thanks" to a recommendation. For instance, adding a small "Not Interested" or swipe-away button on each card. A UX article suggests adding text like "I'm not interested in East Asian crime" when rejecting a category【30†L123-L130】. In effect, the user can thumb-down a genre or topic so those cards won't show again. (Note: studies show this can be hit-or-miss – e.g. YouTube's "Not Interested" alone only removed ~11% of unwanted content【32†L148-L156】 – but blocking whole categories/channels was far more effective (~43% reduction)【32†L148-L156】.)

- **User-built filters:** In practice, users have created custom tools to filter Hotstar content. For example, the "Sift" browser extension automatically hides any Hotstar show with a low IMDb rating【48†L99-L107】. This is exactly the idea of a personal filter: if *you* don't want poor-rated shows, they simply never appear. We can build on that concept: allow the user to define rules (by rating, genre, actor, etc.) and programmatically remove matching titles from all recommendations【48†L99-L107】.

- **Gesture-based UI:** The user mentioned a "left/right swipe" like Tinder. Mobile apps sometimes use swipe gestures for preference. For our web interface, we could implement swipeable cards or buttons ("like/dislike" or "keep/hide") for each suggested title. Each swipe updates the user profile. This interactive feedback loop would feed into our ML model (see below) to refine future recs.

AI/ML Models for Personal Filtering

Building the actual filtering engine can use a mix of methods:

- **Collaborative Filtering (CF):** Traditional CF (matrix-factorization) or neural CF (autoencoders, embeddings) uses past user interactions. For example, a Hotstar rec hackathon solution trained a neural embedding model in Keras/TensorFlow to predict a user's preference for each movie【8†L100-L107】. We could similarly train on any historical data we have (even if it's just the user's own watch/like history) to rank Hotstar titles. Each movie and user gets embedded into a latent space, and similarity/dot-product predicts likes.

- **Content-Based Filtering:** Use movie features (genre, cast, synopsis) to match against user profile. For instance, take each Hotstar movie's synopsis or tags and compute a vector (using TF-IDF, word embeddings, or LLM-based embeddings). Store all title vectors in a database. Then represent the

user by aggregating vectors of liked movies or explicit preferences. For queries, find the nearest neighbors in vector space. A recent example used an LLM (Gemini) to embed every movie synopsis and stored them in Chroma DB for semantic search by "mood"【50†L149-L156】. Similarly, we can convert Hotstar titles to embeddings and filter out any whose similarity falls below a threshold for a user's taste.

- **Hybrid Models:** Combine CF and content. Many systems (Amazon Personalize, Netflix) do this by including content features as side information or by blending two models. We could, for example, use a LightGBM or neural network that takes both a CF score and content-based score, plus genre tags etc., to decide whether to show an item.

- **LLM/Agentic Approaches:** Cutting-edge ideas involve using large language models or multi-agent setups. For instance, one research design simulates multiple "personas" via LLM agents to generate movie suggestions from different angles【54†L37-L41】. In practice, we might build an LLM-driven chatbot that takes in user instructions (e.g. "I like thrillers and hate sports dramas") and uses a Retrieval-Augmented Generation pipeline: it queries our movie metadata (IMDb/OMDb/Watchmode data) and the user's block list to **generate** a tailored list of titles. The LLM could even explain *why* each recommendation fits, adapting to feedback ("No I already saw that one."). Another project built a "mood-based" recommender: it asked the user about mood, then used an LLM to match that mood to movie overviews (using Gemini embeddings)【50†L107-L112】【50†L149-L156】. These agentic systems go beyond static rankings by allowing dialogue and reasoning to filter content.

- **Reinforcement/Bandit:** To continuously improve, one could treat recommendations as a bandit problem: each time the user blocks or likes a movie, update the model's policy to favor similar items. (Although for a single user, simple online learning or periodic retraining might suffice.)

In summary, we can mix traditional ML with modern generative AI: e.g. use a neural CF model as the base ranker, then apply an LLM-based post-filter (via prompt or model) that explicitly removes blocked items and re-scores remaining content according to the user's narrative preferences. Over time, feedback (swipes, thumbs) updates the profile embedding.

Integration Approach and Security Considerations

We must integrate **without hacking Hotstar's core**. The safest approach is a **client-side layer**: e.g. a browser extension or user script that runs inside the user's logged-in Hotstar session. This respects all existing security (the user is already authenticated with JioHotstar) and simply modifies the interface. For example, the Chrome extension "Rating-Finder" already *reads* Hotstar's page and *overlays* IMDb ratings onto each movie tile【39†L66-L73】. We can use the same technique: scan Hotstar's DOM for title elements, compare against the user's block list or filter rules, and dynamically hide or annotate them. In practice this might involve intercepting Hotstar's internal XHR/API calls (via extension background scripts) or simply editing the rendered HTML.

- **Embed or separate UI:** One could also build a completely separate web dashboard. For instance, we fetch Hotstar's catalog (via Watchmode/TMDb as above) and then display only our curated list of recommendations on our site. Each item would have a "Watch on Hotstar" link (a deeplink or URL to

Hotstar's player). The user would still click through to Hotstar for playback. This avoids touching Hotstar's code entirely, but requires the user to use our site instead of the Hotstar home page.

- **Using external APIs:** Both approaches can leverage external data. For each movie, we can call OMDb/TMDb to get ratings, posters, descriptions, etc. This is how "Rating-Finder" works【39†L66-L73】. We can highlight bad content (e.g. red-box low IMDb) and hide it, or use metadata in our ML. We can also use the Watchmode API (or 91mobiles data) to confirm which streaming services a title is on. For tracking watched history, we could use Trakt.tv's API (if the user opts in) or maintain a local database: once the user finishes a movie, mark it as watched so our system never recommends it again.

- **Real-time updates:** Since recommendations should refresh each time the user returns or interacts, the filtering must be dynamic. A browser extension can listen for changes (Hotstar's infinite scroll or AJAX reloads) and re-apply the hide/block logic. A separate web app can poll for new data or refresh on demand. In both cases, there is no need to bypass any Hotstar security layers: we rely on the user's existing login and simply adjust the *what* they see.

- **Legal/TOS:** We must be careful not to break Hotstar's terms. Client-side modifications are usually allowed since the user controls their browser. We should *not* attempt to decrypt or proxy the video stream, and not harvest data at a rate that would violate usage policies. (Notably, commercial "Hotstar scraping APIs" exist【23†L186-L194】 – but those often violate ToS. A browser extension is less likely to draw enforcement since it's user-driven.) All API calls we make to OMDb/TMDb/Watchmode are under their standard terms.

In summary, a feasible architecture is: a browser extension that (1) fetches or listens to Hotstar's content and recommendation data, (2) consults an external recommendation engine (ours) enriched with IMDb/TMDb info, (3) hides any blocked items and re-ranks the rest, and (4) visually presents a clean, filtered homepage. This accomplishes the user's goal — a "fresh" stream of only liked content — without ever reverse-engineering Hotstar's protected streaming pipeline.

Sources: Public reports and docs on Hotstar's tech stack【1†L124-L133】【3†L307-L315】【14†L109-L117】; third-party Hotstar catalog sites【16†L171-L174】 and APIs【24†L37-L45】【26†L279-L287】【23†L186-L194】; UX guidelines on removing unwanted recommendations【30†L100-L107】【30†L123-L130】; real-world examples of filters (Sift extension)【48†L99-L107】 and YouTube study【32†L148-L156】; recent ML/AI recommendation research【8†L100-L107】【50†L107-L112】【50†L149-L156】【54†L37-L41】; and known extension-based integration techniques【39†L66-L73】.
